

ENTITY RELATIONSHIP DIAGRAM (ERD) DESIGN

DOCUMENTATION

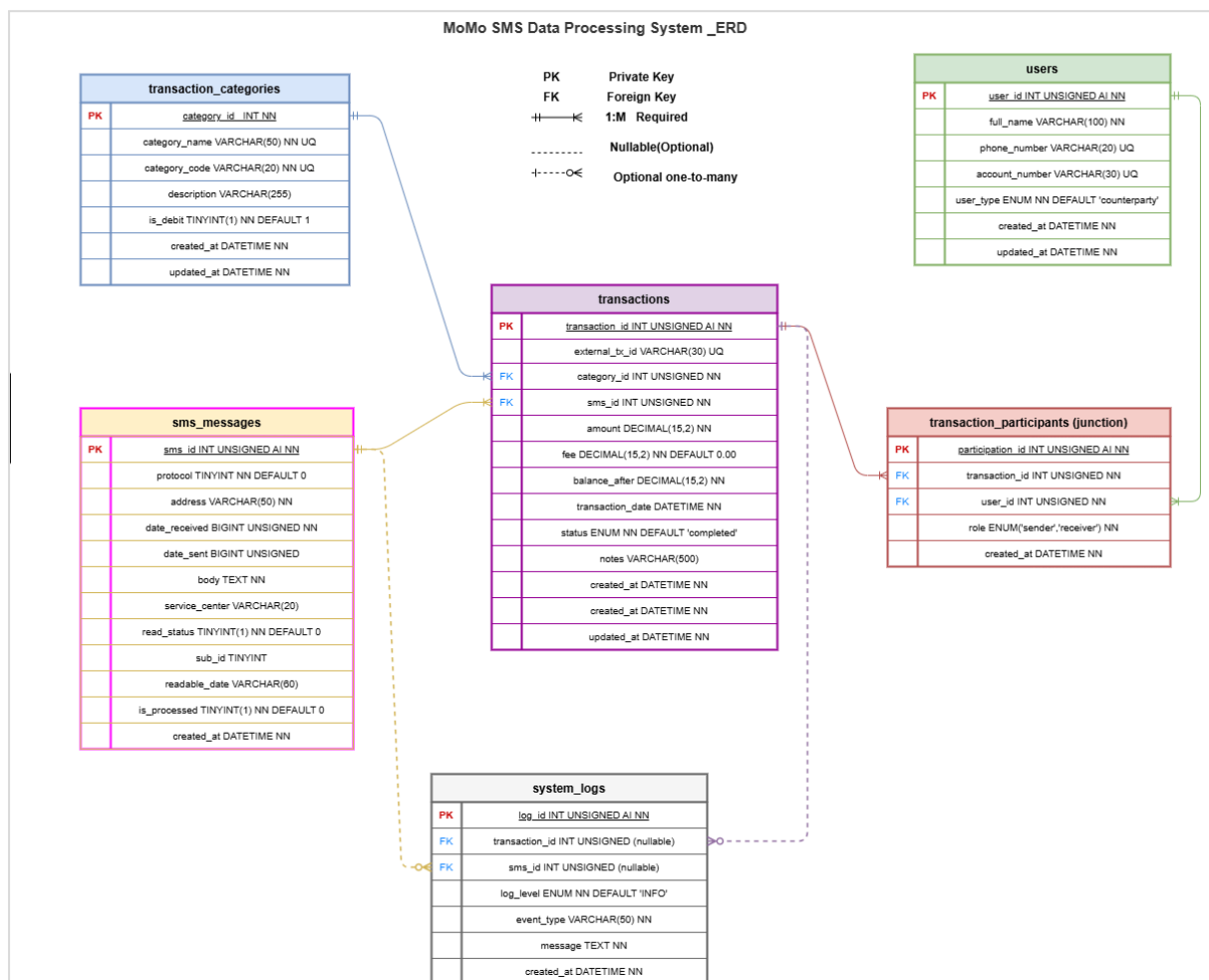
MoMo SMS PROCESSING SYSTEM

Prepared by:

- Panom Michael Makuei
- Nathnael Eticha Ayele
- Success Ituma

Date: 16, May 2026

The database design of the Mobile Money (MoMo) SMS processing system is designed to easily understand transaction information. It is created to make storing, organizing, reading, analyzing and manipulating, and working with the data easy. The key concept of the design was to organize different kinds of information in related tables to maintain the cleanliness, efficiency and also make the system easy to manage.



Entity Overview

transaction_categories

This is a lookup table that stores the 7 types of MoMo transactions identified from analyzing the XML data: Incoming Money, Merchant Payment, Bank Deposit, Mobile Transfer, Airtime Purchase, Cash Withdrawal, and Utility Payment. Each record has a category code suitable for data processing steps in the backend, a human readable category name and a flag field named `is_debit` to identify whether the transaction involved increase or decrease in asset from the wallet. Both category name and category code fields are enforced as unique to prevent duplicate transaction types from being created.

Users

Stores the primary account holder and all other counterparties involved in transaction data. Each user is distinguished by a `user_type` field that accepts four values: `account_holder`, `counterparty`, `merchant`, or `bank`. The `phone_number` field is optional since merchants and banks may not have a phone number available in the SMS data. A regex CHECK constraint is placed to validate that any phone number provided is in a valid format or is null.

sms_messages

Stores raw SMS records directly imported from the XML backup file. Attributes from the xml file are translated into fields such as the date, body, date received and along with the other attributes. It includes a `is_processed` attribute which acts as flag to track whether an sms has been processed by the application or not

Transactions

All the records of a parsed sms are stored here. It stores the amount transacted, the balance after the transaction has occurred, the date it occurred, and any notes extracted from the raw sms by the application. The raw sms is linked to the processed data through the foreign key field `sms_id`, while `category_id` links it back to the category table which contains all the meta data for a transaction type. The CHECK constraints applied to amount, fee, and balance after ensuring no negative value be stored in these columns to prevent any future anomalies and data inconsistencies.

transaction_participants [Junction Table M:N]

Serves to resolve the many-to-many relationship between the users table and transaction table. Contains foreign keys from both the user and transaction tables named `user_id` and `transaction_id`, respectively. Although the combination of these fields can be used as a primary key, the table includes a surrogate key for design consistency named `participation_id`. Furthermore, the combined UNIQUE constraint between `transaction_id` and `role` ensures that a role (sender/receiver) does not appear more than once, in compliance with XML data.

system_logs

Serves to keep a complete audit trail of the entire import, parse, and validation process. Contains a severity level for each log entry (INFO, WARNING, ERROR, or DEBUG) and an event_type limited to five allowed values: IMPORT, PARSE, VALIDATE, CRUD, or SYSTEM. Both foreign keys linked to the transactions and sms_messages tables are nullable and use ON DELETE SET NULL, meaning log entries are kept even if the related transaction or SMS record is deleted. This ensures that the audit history is not lost.

Relationships and Cardinality

From	To	Cardinality	Description
transaction_categories	transactions	1 : M	One category classifies many transactions. A transaction strictly belongs to one category.
sms_messages	transactions	1:M	One SMS record produces one or more parsed transactions. Each transaction traces back to exactly one SMS.
transactions	users	M:N	Resolved through transaction_participants. One user participates in many transactions as sender or receiver. One transaction involves multiple users.
transactions	system_logs	1:0..M	A transaction can have zero or many log entries. FK is nullable so system-level events can be logged without a transaction reference.
sms_messages	system_logs	1:0..M	An SMS record can have zero or many log entries. FK is nullable for the same reason.

Views

There are 3 views created to make the complex queries simple and also support the reporting and API use cases.

vw_transaction_details

This view combines and returns results for transactions, transaction_categories, transaction_participants and users. It displays all the information about each transaction, such as the name of the sender, what kind of sender it is, the receiver name, receiver type, category name, category code, amount, fee, total debit (amount + fee), balance after debit, transaction date, status, and notes. What this really means is that it provides you with the primary view of dashboard queries and API responses that require full transaction details in a single call.

	category_name	is_debit	num_transactions	total_amount_rwf	total_fees_rwf	first_transaction	last_transaction
▶	Bank Deposit	0	1	40000.00	0.00	2024-05-11 18:43:49	2024-05-11 18:43:49
	Merchant Payment	1	6	23400.00	0.00	2024-05-10 16:31:39	2024-05-12 18:08:58
	Mobile Transfer	1	1	10000.00	100.00	2024-05-11 20:34:47	2024-05-11 20:34:47
	Incoming Money	0	1	2000.00	0.00	2024-05-10 16:30:51	2024-05-10 16:30:51
	Airtime Purchase	1	1	2000.00	0.00	2024-05-12 11:41:28	2024-05-12 11:41:28

vw_spending_by_category

This gives the number of transactions, total sum in RWF, total fees paid, and the start and end date of the transactions respectively for each category. Results are sorted by Total Order, which means the highest total transactions will be shown first, allowing you to see which transactions make up the most wallet activity. This is a view to be used for spending analysis and financial summary reporting.

transaction_id	external_tx_id	category_name	category_code	is_debit	amount	fee	total_deducted	balance_after	transaction_date	status	notes	sender_name	sender_type	receiver_name	receiver_type
1	76652021700	Incoming Money	INCOMING	0	2000.00	0.00	2000.00	2000.00	2024-05-10 16:30:51	completed		Jane Smith	merchant	Account Owner	account_holder
2	7321486437	Merchant Payment	MERCHANT_PAY	1	1000.00	0.00	1000.00	1000.00	2024-05-10 16:31:39	completed		Account Owner	account_holder	Jane Smith	merchant
3	BANK_DEP_001	Bank Deposit	BANK_DEP	0	40000.00	0.00	40000.00	40400.00	2024-05-11 18:43:49	completed	Cash deposit via MTN Bank agent 250795963036	MTN Bank	bank	Account Owner	account_holder
4	17818959211	Merchant Payment	MERCHANT_PAY	1	2000.00	0.00	2000.00	38400.00	2024-05-11 18:48:42	completed		Account Owner	account_holder	Samuel Carter	counterparty
5	MOB_TRF_001	Mobile Transfer	MOB_TRANSFER	1	10000.00	100.00	10100.00	28300.00	2024-05-11 20:34:47	completed		Account Owner	account_holder	Account Owner	account_holder
6	13913173274	Airtime Purchase	AIRTIME	1	2000.00	0.00	2000.00	25280.00	2024-05-12 11:41:28	completed		Account Owner	account_holder	Account Owner	account_holder
7	45434420466	Merchant Payment	MERCHANT_PAY	1	10900.00	0.00	10900.00	14380.00	2024-05-12 13:26:13	completed		Account Owner	account_holder	Jane Smith	merchant
8	82113964658	Merchant Payment	MERCHANT_PAY	1	3500.00	0.00	3500.00	10880.00	2024-05-12 13:34:25	completed		Account Owner	account_holder	Alex Doe	merchant
9	26614842768	Merchant Payment	MERCHANT_PAY	1	1000.00	0.00	1000.00	9880.00	2024-05-12 17:58:15	completed		Account Owner	account_holder	Robert Brown	merchant
10	70497610538	Merchant Payment	MERCHANT_PAY	1	5000.00	0.00	5000.00	4880.00	2024-05-12 18:08:58	completed		Account Owner	account_holder	Linda Green	merchant

vw_user_transaction_history

This maps the user, transaction_participants, transactions, and transaction_categories together and generates a comprehensive transaction history for each user. Each row displays the user, the role of the user in the transaction (sender or receiver), the external transaction ID, category name, transaction amount, fee, transaction date, and transaction status. The results are displayed in reverse-chronological order, from the most recent to the oldest. This view is built for per user account history queries and individual analytics.

Result Grid										
		Filter Rows:		Exports:		Wrap Cell Content:				
	user_id	full_name	user_type	role	external_tx_id	category_name	amount	fee	transaction_date	status
▶	1	Account Owner	account_holder	sender	70497610538	Merchant Payment	5000.00	0.00	2024-05-12 18:08:58	completed
	6	Linda Green	merchant	receiver	70497610538	Merchant Payment	5000.00	0.00	2024-05-12 18:08:58	completed
	1	Account Owner	account_holder	sender	26614842768	Merchant Payment	1000.00	0.00	2024-05-12 17:58:15	completed
	5	Robert Brown	merchant	receiver	26614842768	Merchant Payment	1000.00	0.00	2024-05-12 17:58:15	completed
	1	Account Owner	account_holder	sender	82113964658	Merchant Payment	3500.00	0.00	2024-05-12 13:34:25	completed
	4	Alex Doe	merchant	receiver	82113964658	Merchant Payment	3500.00	0.00	2024-05-12 13:34:25	completed
	1	Account Owner	account_holder	sender	45434420466	Merchant Payment	10900.00	0.00	2024-05-12 13:26:13	completed
	2	Jane Smith	merchant	receiver	45434420466	Merchant Payment	10900.00	0.00	2024-05-12 13:26:13	completed
	1	Account Owner	account_holder	sender	13913173274	Airtime Purchase	2000.00	0.00	2024-05-12 11:41:28	completed
	1	Account Owner	account_holder	sender	MOB_TRF_001	Mobile Transfer	10000.00	100.00	2024-05-11 20:34:47	completed
	3	Samuel Carter	counterparty	receiver	MOB_TRF_001	Mobile Transfer	10000.00	100.00	2024-05-11 20:34:47	completed
	1	Account Owner	account_holder	sender	17818959211	Merchant Payment	2000.00	0.00	2024-05-11 18:48:42	completed
	3	Samuel Carter	counterparty	receiver	17818959211	Merchant Payment	2000.00	0.00	2024-05-11 18:48:42	completed
	7	MTN Bank	bank	sender	BANK_DEP_001	Bank Deposit	40000.00	0.00	2024-05-11 18:43:49	completed
	1	Account Owner	account_holder	receiver	BANK_DEP_001	Bank Deposit	40000.00	0.00	2024-05-11 18:43:49	completed
	1	Account Owner	account_holder	sender	73214484437	Merchant Payment	1000.00	0.00	2024-05-10 16:31:39	completed
	2	Jane Smith	merchant	receiver	73214484437	Merchant Payment	1000.00	0.00	2024-05-10 16:31:39	completed
	2	Jane Smith	merchant	sender	76662021700	Incoming Money	2000.00	0.00	2024-05-10 16:30:51	completed
	1	Account Owner	account_holder	receiver	76662021700	Incoming Money	2000.00	0.00	2024-05-10 16:30:51	completed

Design Rationale and Justifications

An important problem faced was that raw SMS text messages cannot be queried, analysed and extended as is. The schema converts the unstructured SMS messages from MTN MoMo XML file into a fully normalized relational database, which allows meaningful financial analysis and makes it manageable.

The design starts with a small lookup table called `transaction_categories`, which classifies transactions into types based on the information that comes directly from the XML SMS bodies. Putting these types in a specific table keeps them from being duplicated; they have a consistent name; and they are easily filtered on in all queries by transaction type. If this table did not exist, then the name of the category would appear in each transaction row (first normal form violation).

Users table is used to keep the users data separate from transaction data. This is based on the concept of storing information once in a file, and can be accessed many times. Instead of having the sender's name in each transaction record, we store the user only once and link to it. This also enables updates of user information, such as phone numbers, without impacting transactions.

The `sms_messages` table was intentionally added to ensure that the raw data from the XML file is preserved in an identical format. This is important for auditability as in the event of any dispute, the original SMS text remains in the database. Each transaction record is associated with the SMS it was parsed from, leaving a clear chain of evidence.

The `transaction_participants` table was designed to correctly handle the many to many relationship between the transactions and the users, as part of the assignment. Instead of two separate foreign key columns in the transactions table storing the sender and receiver, a junction table can be used to make the relationship fully flexible. It also implies that the table schema of a transaction can be modified without affecting the transactions table.

To meet processing pipeline operational needs, the system_logs table was added. There will be issues when many sms of about 1,693 are being imported and parsed: unrecognized formats, missing transaction IDs, validation issues. All of these events are recorded in the log table with a time stamp, severity, and a reference to the SMS or transaction in question, and the system is debuggable and transparent.

The six tables together are in third normal form: that is, no non-key attribute is dependent on another non-key attribute. The schema is designed to be scalable and will not need to be structurally changed when more SMS information is added to the processing for future cycles.

Data Dictionary

The following section documents every table and column in the database. For each column the data type, nullability, key type, and a full description of its purpose are provided.

transaction_categories

Lookup table for all MoMo transaction types. This table rarely changes — it holds the 7 transaction types seeded from analysis of the XML data.

Column Name	Data Type	Nullable	Key	Description
category_id	INT UNSIGNED	NOT NULL	PK	Auto-incremented primary key. Uniquely identifies each transaction type.
category_name	VARCHAR(50)	NOT NULL	UNIQUE	Name of the transaction type e.g. Incoming Money, Merchant Payment.
category_code	VARCHAR(20)	NOT NULL	UNIQUE	Short code used in processing logic e.g. INCOMING, MERCHANT_PAY, BANK_DEP.
description	VARCHAR(255)	NOT NULL		Optional full description of what this transaction type represents.
is_debit	TINYINT(1)	NOT		Indicates direction of money: 1 =

		NULL		money leaves wallet (debit), 0 = money enters wallet (credit). CHECK: must be 0 or 1.
created_at	DATETIME	NOT NULL		Timestamp automatically set when the record was first created.
updated_at	DATETIME	NOT NULL		Timestamp automatically updated whenever this record is modified.

users

Stores the primary account holder and all counterparties encountered in the SMS transaction data.

Each user appears only once regardless of how many transactions they are involved in.

Column Name	Data Type	Nullable	Key	Description
user_id	INT UNSIGNED	NOT NULL	PK	Auto-incremented primary key. Uniquely identifies each user.
full_name	VARCHAR(100)	NOT NULL		Full name of the user exactly as it appears in the SMS body text.
phone_number	VARCHAR(20)	NULL		International phone number e.g. +250791234567. Optional, merchants and banks may not have one. CHECK: must match international format if provided.
user_type	ENUM	NOT NULL		Role of the user in the system. Accepted values: account_holder, counterparty, merchant, bank.
created_at	DATETIME	NOT NULL		Timestamp automatically set when the record was first created.
updated_at	DATETIME	NOT NULL		Timestamp automatically updated whenever this record is modified.

sms_messages

Raw SMS records imported from the Android XML backup. Every XML attribute is preserved. This is the source of truth for all transaction data.

Column Name	Data Type	Nullabl e	Ke y	Description
sms_id	INT UNSIGNED	NOT NULL	PK	Auto-incremented primary key. Uniquely identifies each SMS record.
protocol	TINYINT	NOT NULL		SMS protocol code from the XML attribute. Always 0 for MoMo messages.
address	VARCHAR(50)	NOT NULL		Sender address from the XML. Always M-Money for MoMo SMS messages.
date_received	BIGINT UNSIGNED	NOT NULL		Unix timestamp in milliseconds recording when the SMS was received. Sourced from the XML date attribute.
date_sent	BIGINT UNSIGNED	NULL		Unix timestamp in milliseconds recording when the SMS was sent by the service centre. Sourced from XML date_sent attribute.
body	TEXT	NOT NULL		The full raw SMS body text. Contains all transaction details in unstructured form.
service_center	VARCHAR(20)	NULL		The SMS service centre number that delivered the message e.g. +250788110381.
read_status	TINYINT(1)	NOT NULL		Indicates whether the SMS has been read. 1 = read, 0 = unread. CHECK: must be 0 or 1.
sub_id	TINYINT	NULL		SIM card subscription ID recorded by the Android device.
readable_date	VARCHAR(60)	NULL		Human-readable date string as provided in the XML e.g. 10 May 2024 4:30:58 PM.
is_proccessed	TINYINT(1)	NOT NULL		Processing status flag. 1 = this SMS has been parsed into the transactions table. 0 = pending processing. CHECK: must be 0 or 1.

created_at	DATETIME	NOT NULL		Timestamp automatically set when the record was inserted into this table.
------------	----------	-------------	--	---

transactions

Parsed and structured financial records. Each row represents one complete financial event extracted from an SMS. This is the primary analytical table, most reporting queries run against this table.

Column Name	Data Type	Nullable	Key	Description
transaction_id	INT UNSIGNED	NOT NULL	PK	Auto-incremented surrogate primary key. Uniquely identifies each transaction.
external_tx_id	VARCHAR(30)	NULL	UNIQUE	The TxId or Financial Transaction Id extracted from the SMS body. NULL for transaction types that have no standard ID such as bank deposits.
category_id	INT UNSIGNED	NOT NULL	FK	Foreign key to transaction_categories. Identifies the type of transaction. ON UPDATE CASCADE.
sms_id	INT UNSIGNED	NULL	FK	Foreign key to the source SMS record in sms_messages. Nullable: if the SMS is deleted, this is set to NULL but the transaction is preserved. ON DELETE SET NULL.
amount	DECIMAL(15,2)	NOT NULL		Transaction amount in Rwandan Franc (RWF). CHECK: must be greater than 0.
fee	DECIMAL(15,2)	NOT NULL		Transaction fee charged in RWF. Defaults to 0.00 for fee-free transactions. CHECK: must be 0 or greater.
balance_after	DECIMAL(15,2)	NOT NULL		The wallet balance immediately after this transaction was completed. CHECK: must be 0 or greater.
transaction_date	DATETIME	NOT NULL		The actual timestamp of the financial event as stated in the SMS body text.

status	ENUM	NOT NULL		The current state of the transaction. Accepted values: completed, pending, failed, reversed.
notes	VARCHAR(500)	NULL		Optional sender message or processing note attached to the transaction.
created_at	DATETIME	NOT NULL		Timestamp automatically set when the record was inserted.
updated_at	DATETIME	NOT NULL		Timestamp automatically updated whenever this record is modified.

transaction_participants

Junction table that resolves the M:N relationship between transactions and users. One user participates in many transactions, and one transaction involves multiple users. This table makes that relationship explicit through a role column.

Column Name	Data Type	Nullable	Key	Description
participation_id	INT UNSIGNED	NOT NULL	PK	Auto-incremented surrogate primary key.
transaction_id	INT UNSIGNED	NOT NULL	FK	Foreign key to transactions. ON DELETE CASCADE — when a transaction is deleted all its participant rows are automatically removed.
user_id	INT UNSIGNED	NOT NULL	FK	Foreign key to users. Identifies which user participated in this transaction. ON UPDATE CASCADE.
role	ENUM('sender','receiver')	NOT NULL		The role of this user in the transaction. UNIQUE constraint on (transaction_id, role) ensures only one sender and one receiver per transaction.
created_at	DATETIME	NOT NULL		Timestamp automatically set when the participant record was created.

system_logs

Full audit trail for all data processing events. Records every import, parse, validation, CRUD, and system event with a severity level and detailed message. Both foreign keys are nullable so system-level events can be logged without referencing a specific transaction or SMS.

Column Name	Data Type	Nullable	Key	Description
log_id	INT UNSIGNED	NOT NULL	PK	Auto-incremented surrogate primary key.
transaction_id	INT UNSIGNED	NULL	FK	Optional foreign key to transactions. NULL for system-level events not tied to a transaction. ON DELETE SET NULL.
sms_id	INT UNSIGNED	NULL	FK	Optional foreign key to sms_messages. NULL when not applicable. ON DELETE SET NULL.
log_level	ENUM('INFO','WARNING','ERROR','DEBUG')	NOT NULL		Severity level of the log entry: INFO for normal events, WARNING for non-critical issues, ERROR for failures, DEBUG for diagnostic information.
event_type	VARCHAR(50)	NOT NULL		The processing stage that generated this entry. CHECK constraint restricts to: IMPORT, PARSE, VALIDATE, CRUD, SYSTEM.
message	TEXT	NOT NULL		Full detailed message describing what occurred during processing.
created_at	DATETIME	NOT NULL		Timestamp automatically set when the log entry was recorded.

Sample Queries Demonstrating Database Functionality

After running the full database_setup.sql script, here are the queries that were executed in MySQL Workbench. They show all 4 crud operations on all the tables of the database (Create, Read, Update and Delete).

READ Queries

READ 1: All Transactions Ordered by Date

This displays the complete transaction history for the account. This report will show the account owner's full transaction history.

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	external_tx_id	category_name	amount	fee	balance_after	transaction_date	status
▶	76662021700	Incoming Money	2000.00	0.00	2000.00	2024-05-10 16:30:51	completed
	73214484437	Merchant Payment	1000.00	0.00	1000.00	2024-05-10 16:31:39	completed
	BANK_DEP_001	Bank Deposit	40000.00	0.00	40400.00	2024-05-11 18:43:49	completed
	17818959211	Merchant Payment	2000.00	0.00	38400.00	2024-05-11 18:48:42	completed
	MOB_TRF_001	Mobile Transfer	10000.00	100.00	28300.00	2024-05-11 20:34:47	completed
	13913173274	Airtime Purchase	2000.00	0.00	25280.00	2024-05-12 11:41:28	completed
	45434420466	Merchant Payment	10900.00	0.00	14380.00	2024-05-12 13:26:13	completed
	82113964658	Merchant Payment	3500.00	0.00	10880.00	2024-05-12 13:34:25	completed
	26614842768	Merchant Payment	1000.00	0.00	9880.00	2024-05-12 17:58:15	completed
	70497610538	Merchant Payment	5000.00	0.00	4880.00	2024-05-12 18:08:58	completed

READ 2: Account Owner Full Transaction History

Fetches all transactions for the main account holder (user_id = 1) by merging transaction_participants, transactions, and transaction_categories. The output displays the user's role on each transaction (sender/receiver), amount, fee, balance after and date. This is an example of a multi-table JOIN with filter applied, for one particular user.

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	external_tx_id	category_name	role	amount	fee	balance_after	transaction_date
▶	76662021700	Incoming Money	receiver	2000.00	0.00	2000.00	2024-05-10 16:30:51
	73214484437	Merchant Payment	sender	1000.00	0.00	1000.00	2024-05-10 16:31:39
	BANK_DEP_001	Bank Deposit	receiver	40000.00	0.00	40400.00	2024-05-11 18:43:49
	17818959211	Merchant Payment	sender	2000.00	0.00	38400.00	2024-05-11 18:48:42
	MOB_TRF_001	Mobile Transfer	sender	10000.00	100.00	28300.00	2024-05-11 20:34:47
	13913173274	Airtime Purchase	sender	2000.00	0.00	25280.00	2024-05-12 11:41:28
	45434420466	Merchant Payment	sender	10900.00	0.00	14380.00	2024-05-12 13:26:13
	82113964658	Merchant Payment	sender	3500.00	0.00	10880.00	2024-05-12 13:34:25
	26614842768	Merchant Payment	sender	1000.00	0.00	9880.00	2024-05-12 17:58:15
	70497610538	Merchant Payment	sender	5000.00	0.00	4880.00	2024-05-12 18:08:58

READ 3: Total Spending Grouped by Category

Groups all debit transactions by category, displays the number of transactions, amount spent and amount of fees paid by category. The results are listed in descending order by the sum. This query returns the sum and count of all the transactions that are outgoing (is_debit = 'true') and groups them all by transaction.GUID. This query groups all the transactions by transaction.GUID, and includes the sum and count of all the transactions that are going out (is_debit = 'true').

Result Grid  Filter Rows: Export:  Wrap Cell Content: 				
	category_name	num_transactions	total_rwf	total_fees
▶	Merchant Payment	6	23400.00	0.00
	Mobile Transfer	1	10000.00	100.00
	Airtime Purchase	1	2000.00	0.00

READ 4: Transactions Above 5,000 RWF

Gets all transactions with an amount greater than 5000 RWF, sorted by amount (descending). Use this query to filter on a financial value, to find high value transactions.

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
	external_tx_id	amount	fee	balance_after	transaction_date
▶	BANK_DEP_001	40000.00	0.00	40400.00	2024-05-11 18:43:49
	45434420466	10900.00	0.00	14380.00	2024-05-12 13:26:13
	MOB_TRF_001	10000.00	100.00	28300.00	2024-05-11 20:34:47

READ 5: Full Transaction Detail View

Queries the vw_transaction_details view to list full details of transactions including sender name, receiver name, category, amount, fee, total deducted and balance after. This shows how to use a pre-built view to make complex queries on multiple tables easier to write as a single select statement.

transaction_id	external_tx_id	category_name	category_code	is_debit	amount	fee	total_deducted	balance_after	transaction_date	status	notes	sender_name	sender_type	receiver_name	receiver_type
1	76662021700	Incoming Money	INCOMING	0	2000.00	0.00	2000.00	2000.00	2024-05-10 16:30:51	completed		Jane Smith	merchant	Account Owner	account_holder
2	7321484407	Merchant Payment	MERCHANT_PAY	1	1000.00	0.00	1000.00	1000.00	2024-05-10 16:31:39	completed		Account Owner	account_holder	Jane Smith	merchant
3	BANK_DEP_001	Bank Deposit	BANK_DEP	0	40000.00	0.00	40000.00	40400.00	2024-05-11 18:43:49	completed	Cash deposit via MTN Bank agent 250795963036	MTN Bank	bank	Account Owner	account_holder
4	17818959211	Merchant Payment	MERCHANT_PAY	1	2000.00	0.00	2000.00	38400.00	2024-05-11 18:48:42	completed		Account Owner	account_holder	Samuel Carter	counterparty
5	MOB_TRF_001	Mobile Transfer	MOB_TRANSFER	1	10000.00	100.00	10100.00	28300.00	2024-05-11 20:34:47	completed		Account Owner	account_holder	Account Owner	account_holder
6	13913173274	Airtime Purchase	AIRTIME	1	2000.00	0.00	2000.00	25280.00	2024-05-12 11:41:28	completed		Account Owner	account_holder	Account Owner	account_holder
7	45434420466	Merchant Payment	MERCHANT_PAY	1	10900.00	0.00	10900.00	14380.00	2024-05-12 13:26:13	completed		Account Owner	account_holder	Jane Smith	merchant
8	8211964658	Merchant Payment	MERCHANT_PAY	1	3500.00	0.00	3500.00	10880.00	2024-05-12 13:34:25	completed		Account Owner	account_holder	Alex Doe	merchant
9	26614842768	Merchant Payment	MERCHANT_PAY	1	1000.00	0.00	1000.00	9880.00	2024-05-12 17:58:15	completed		Account Owner	account_holder	Robert Brown	merchant
10	70497610538	Merchant Payment	MERCHANT_PAY	1	5000.00	0.00	5000.00	4880.00	2024-05-12 18:08:58	completed		Account Owner	account_holder	Linda Green	merchant

UPDATE Queries

UPDATE 1: Reverse a Transaction

Simulates reversing a transaction to change its status from completed to reversed. A SELECT query is then executed immediately after to ensure that the change was made. The transaction is then rolled back to completed. This shows full update-verify restore cycle and shows that the status ENUM constraint works as expected.

external_tx_id	status	updated_at
70497610538	completed	2026-05-18 14:57:42

external_tx_id	status	updated_at
70497610538	reversed	2026-05-18 14:55:58

#	Time	Action	Message	Duration / Fetch
1	14:55:47	CREATE DATABASE IF NOT EXISTS momo_sms_...	1 row(s) affected, 1 warning(s): 1007 Can't create d...	0.015 sec
2	14:55:47	USE momo_sms_db	0 row(s) affected	0.000 sec
3	14:55:47	CREATE TABLE transaction_categories (catego...	Error Code: 1050. Table 'transaction_categories' alr...	0.063 sec
4	14:55:58	UPDATE transactions SET status = 'reversed', upd...	1 row(s) affected Rows matched: 1 Changed: 1 W...	0.031 sec
5	14:56:23	SELECT external_tx_id, status, updated_at FROM t...	1 row(s) returned	0.000 sec / 0.000 sec
6	14:57:42	UPDATE transactions SET status = 'completed', up...	1 row(s) affected Rows matched: 1 Changed: 1 W...	0.000 sec
7	15:00:30	SELECT external_tx_id, status, updated_at FROM t...	1 row(s) returned	0.000 sec / 0.000 sec

UPDATE 2: Add a Missing Phone Number

Updates a user record (Jane Smith) with a new phone number that wasn't available at the time of the initial data insert. This shows how to update a nullable field, and that a valid international number is acceptable for the phone number format CHECK constraint.

1 • UPDATE users SET phone_number = '+250789000001', updated_at = NOW() WHERE full_name = 'Jane Smith' AND user_type = 'merchant';

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	15:15:22	UPDATE users SET phone_number = '+250789000001', updated_at = NOW() WH...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 sec

INSERT Query

INSERT: Manual Audit Log Entry

Adds a new log entry in system_logs that the administrator manually re-verified a transaction. This is an example of inserting into the system_logs table, using a transaction foreign key reference, and illustrates the event_type check constraint with a valid value.

1 15:35:53 INSERT INTO system_logs (transaction_id, sms_id, log_level, event_type, message) VALUES (1, NULL, 'INFO', 'VALIDATE', 'Manual a...

Output

Action Output

#	Time	Action	Message
1	15:35:53	INSERT INTO system_logs (transaction_id, sms_id, log_level, event_type, message) VALUES (1, NULL, 'INFO', 'VALIDATE', 'Manual audit: transaction 76662021700 re-verified by admin on 2024-05-15.')	1 row(s) affected

DELETE Query

DELETE: Remove Stale Debug Logs

Cleans up logs of DEBUG level older than 30 days during regular log maintenance. This is an example of a conditional DELETE on both a log level filter and a date based filter. You can execute it over and over again without problems and if there are no rows matching it, it will not change.

1 15:39:52 DELETE FROM system_logs WHERE log_level = 'DEBUG' AND created_... 0 row(s) affected 0.016 sec

2 15:40:48 DELETE FROM system_logs WHERE log_level = 'DEBUG' AND created_... 0 row(s) affected 0.016 sec

3 15:40:49 DELETE FROM system_logs WHERE log_level = 'DEBUG' AND created_... 0 row(s) affected 0.000 sec

4 15:40:50 DELETE FROM system_logs WHERE log_level = 'DEBUG' AND created_... 0 row(s) affected 0.000 sec

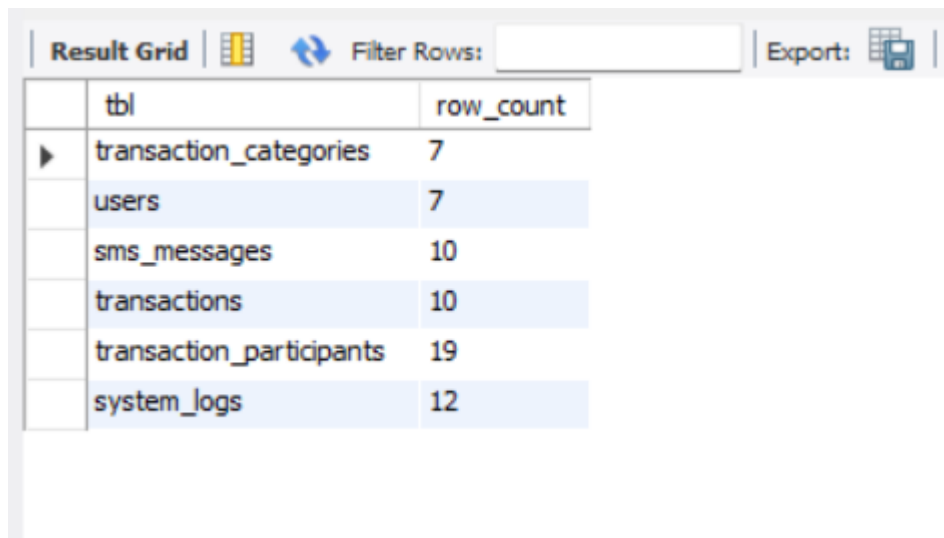
Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	15:39:52	DELETE FROM system_logs WHERE log_level = 'DEBUG' AND created_...	0 row(s) affected	0.016 sec
2	15:40:48	DELETE FROM system_logs WHERE log_level = 'DEBUG' AND created_...	0 row(s) affected	0.016 sec
3	15:40:49	DELETE FROM system_logs WHERE log_level = 'DEBUG' AND created_...	0 row(s) affected	0.000 sec
4	15:40:50	DELETE FROM system_logs WHERE log_level = 'DEBUG' AND created_...	0 row(s) affected	0.000 sec

Row Count Verification

A final verification query is provided to show the number of rows in the six tables in a single result, via UNION ALL. It verifies that all sample data has been added properly, and that the database is in the desired condition.



	tbl	row_count
▶	transaction_categories	7
	users	7
	sms_messages	10
	transactions	10
	transaction_participants	19
	system_logs	12

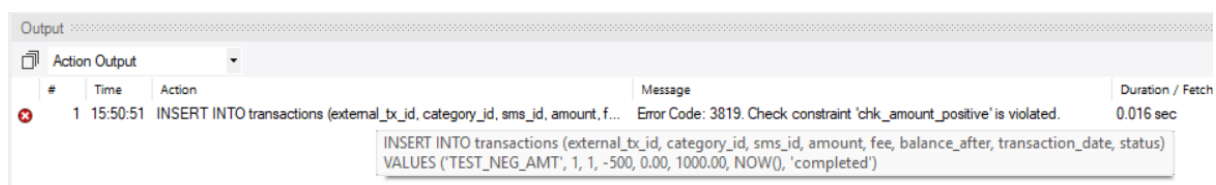
Unique Rules Added to Enhance Security and Accuracy

The database applies a set of constraints across all six tables, to ensure that the data is valid, duplicate-free and meets business rules. The constraints were selected depending on the real-life processes of MoMo transactions.

CHECK Constraints

CHECK (amount > 0): transactions

Makes sure that all transactions are positive and in excess of zero. If the amount is zero or even negative, this means the transaction has no financial purpose and is probably corrupt or malicious data. This constraint will prevent any 0 or lower value for amount being inserted or changed.



#	Time	Action	Message	Duration / Fetch
1	15:50:51	INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, f...	Error Code: 3819. Check constraint 'chk_amount_positive' is violated.	0.016 sec

INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, fee, balance_after, transaction_date, status) VALUES ('TEST_NEG_AMT', 1, 1, -500, 0.00, 1000.00, NOW(), 'completed')

CHECK (fee >= 0): transactions

Maintains that transaction fees do not become negative. A transaction fee of 0 is acceptable and typical of MoMo while having a negative transaction fee would be useless. The constraint prevents invalid entries into the fee column, but does not prevent fee-free transactions.

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	15:52:18	INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, f...	Error Code: 3819. Check constraint 'chk_fee_non_negative' is violated.	0.000 sec
INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, fee, balance_after, transaction_date, status) VALUES ('TEST_NEG_FEE', 1, 1, 1000.00, -100, 5000.00, NOW(), 'completed')				

CHECK (balance_after >= 0): transactions

Does not allow the wallet balance to drop below zero after making a transaction. In MoMo it is not possible to have a negative wallet balance and such a data error would be indicated. This restriction is a last line of defense to the balance field.

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	15:54:01	INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, f...	Error Code: 3819. Check constraint 'chk_balance_non_negative' is violated.	0.016 sec
INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, fee, balance_after, transaction_date, status) VALUES ('TEST_NEG_BAL', 1, 1, 1000.00, 0.00, -1000.00, NOW(), 'completed')				

CHECK (is_debit IN (0, 1)):transaction_categories

Only allows binary values for is_debit. Only 0 (credit) and 1 (debit) have meaning in this flag because it indicates the direction of the money. Other integers would make spending analysis queries illogical.

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	15:55:33	INSERT INTO transaction_categories (category_name, category_code, is_...	Error Code: 3819. Check constraint 'chk_is_debit' is violated.	0.000 sec
INSERT INTO transaction_categories (category_name, category_code, is_debit) VALUES ('Bad Category', 'BAD_CAT', 5)				

CHECK (read_status IN (0, 1)): sms_messages

This query is used to verify the status of the SMS messages. This is a query used to check the status of SMS messages (0, 1).

Validates read_status field to only allow 0 (unread) or 1 (read). This ensures boolean integrity of a field that is used for SMS processing decisions.

Output			
Action Output			
#	Time	Action	Message
1	19:00:23	INSERT INTO sms_messages (protocol, address, date_received, body, read_status, is_processed) VALUES (0, 'MoMo', 17150000000, 'test body', 5, 0)	Error Code: 3819. Check constraint 'chk_read_status' is violated.

CHECK (is_processed IN (0, 1)): sms_messages

Only accepts 0 (pending) or 1 (done) for the is_processed flag. This field is used to select which SMS records should be picked up by the processing pipeline. Bad values would skip or double process the records..

Output			
Action Output			
#	Time	Action	Message
1	15:58:35	INSERT INTO sms_messages (protocol, address, date_received, body, read_status, is_processed) VALUES (0, 'MoMo', 1715000000000, 'test body', 0, 2)	Error Code: 3819. Check constraint 'chk_is_processed' is violated.

CHECK (phone_number REGEXP ...): users

Validates that a phone number entered is in the international format (optional + character followed by 9-15 digits). NULL also is permitted because many counterparties and merchants don't have a phone number in the SMS data. This will keep any numbers that are poorly formatted from being stored, but will also allow for optional fields.

Output			
Action Output			
#	Time	Action	Message
1	16:00:29	INSERT INTO users (full_name, phone_number, user_type) VALUES ('Test User', 'not-a-number', 'counterparty')	Error Code: 3819. Check constraint 'chk_phone_format' is violated.

CHECK (event_type IN (...)): system_logs

Limits the types of processing stages that can be entered into the event_type column to five known processing stages: IMPORT, PARSE, VALIDATE, CRUD, and SYSTEM. This will stop arbitrary strings from being added to the audit log and allow logs to be reliably searched by stage.

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	16:01:39	INSERT INTO system_logs (log_level, event_type, message) VALUES (IN...	Error Code: 3819. Check constraint 'chk_event_type' is violated.	0.000 sec
INSERT INTO system_logs (log_level, event_type, message) VALUES ('INFO', 'UNKNOWN', 'This should fail')				

UNIQUE Constraints

UNIQUE (category_code) and UNIQUE (category_name): transaction_categories

Does not allow multiple transaction types to be added. In the case of two categories with the same name or code, queries made using category filters would produce ambiguous results. These are two special constraints to keep the look-up table clean and reliable.

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	16:02:59	INSERT INTO transaction_categories (category_name, category_code, is_...	Error Code: 1062. Duplicate entry 'Incoming Money' for key transaction_ca...	0.047 sec
INSERT INTO transaction_categories (category_name, category_code, is_debit) VALUES ('Incoming Money', 'DUPLICATE_CODE', 0)				

UNIQUE (external_tx_id): transactions

Does not allow the same MoMo transaction ID to be added to the database twice. This is the most crucial data accuracy rule in the system. If it were not there, the same XML file would result in two sets of financial records. The external_tx_id is optional to allow transactions that do not have a standard identifier, but if present then it must be unique across all transactions.

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	16:05:21	INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, f...	Error Code: 1062. Duplicate entry '76662021700' for key transactions.uq...	0.016 sec
INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, fee, balance_after, transaction_date, status) VALUES ('76662021700', 1, 1, 500.00, 0.00, 1000.00, NOW(), 'completed')				

UNIQUE (transaction_id, role): transaction_participants

Implements that a transaction can have only one sender and one receiver. Otherwise, the participation data would be corrupted and queries depending on the one-to-one sender/receiver relationship would be broken if two sender rows could be inserted for the same transaction.

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	16:06:32	INSERT INTO transaction_participants (transaction_id, user_id, role) VALU...	Error Code: 1062. Duplicate entry '1-sender' for key transaction_participant...	0.016 sec
INSERT INTO transaction_participants (transaction_id, user_id, role) VALUES (1, 3, 'sender')				

FOREIGN KEY Constraints

ON DELETE CASCADE: transaction_participants

All transaction_participants rows are deleted along with the row being deleted from the transactions table. This ensures that there are never orphaned participant records in the junction table, and maintains consistency in the database.

Result Grid					
participation_id	transaction_id	user_id	role	created_at	
18	10	1	sender	2026-05-16 16:57:06	
19	10	6	receiver	2026-05-16 16:57:06	
NULL	NULL	NULL	NULL	NULL	

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	19:03:07	SELECT * FROM transaction_participants WHERE transaction_id = 10	2 row(s) returned	

ON DELETE SET NULL: transactions (sms_id)

If an SMS record in the sms_messages table is deleted, then the sms_id column in any associated transaction is set to NULL, but the transaction itself is not deleted. This will maintain the financial record, without tying it to the raw SMS. Even if the source SMS is cleaned up, financial history is never lost.

ON DELETE SET NULL: system_logs

Any logging entries that refer to a transaction or an SMS record are not deleted when the record is deleted, but instead have their FK field set to NULL. This means that the complete audit history is always maintained irrespective of what may happen to the source records.

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	16:08:02	INSERT INTO transactions (external_tx_id, category_id, sms_id, amount, fee, balance_after, transaction_date, status) VALUES (TEST...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('momo_sms_db', 'transactions', CONSTRAINT fk_tr...	0.015 sec
Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('momo_sms_db', 'transactions', CONSTRAINT fk_transactions_category' FOREIGN KEY ('category_id') REFERENCES 'transaction_categories' ('category_id') ON UPDATE CASCADE)				